

Accurate Bounded Model Checking for OSEK/VDX Applications in CBMC

CBMC ISR/OSEK Research Branch

June 20, 2026

Abstract

OSEK/VDX applications consist of multiple tasks, a deterministic scheduler, a ready queue, and standardized application interfaces (APIs). If existing model checking methods for concurrent software are applied directly to such applications, a large number of interleavings of tasks may be checked although the running task is explicitly determined by the scheduler. The result is an approximate verification in which unnecessary interleavings of tasks may lead to a spurious bug. This report describes a CBMC-based bounded model checking flow that takes the behaviours of OSEK/VDX scheduler into account and reduces unnecessary interleavings before source-to-source instrumentation.

The first part uses OIL configuration and scheduler-related APIs so that task states, the ready queue, and task execution order follow the OSEK/VDX model during verification. The second part computes a manifest of candidate program points whose accesses may conflict with globals written by the interruption code, and then inserts guarded nondeterministic calls only at those points. On the public benchmark subset in this repository, the checked-in evidence contains 43 measured cases: 17 same-outcome comparable cases, 10 cases audited as `improved_correct`, 6 `no_injection_false` cases, 3 `needs_manual_review` cases, and 7 `pending_manual_review` cases. Representative public cases show that the method can remove unnecessary interleavings that produce spurious bugs and can also expose error traces that are not reached by the stock baseline. The added accuracy costs extra analysis and rebuild time, but it yields a more accurate bounded model checking workflow for OSEK/VDX applications.

1 Introduction

OSEK/VDX is an operating-system standard for vehicle-mounted software [1]. An application developed on OSEK/VDX OS consists of multiple tasks. To concurrently execute multiple tasks on one CPU, OSEK/VDX scheduler uses task states, priorities, and a ready queue to determine the running task. In contrast with many forms of concurrent software in which the running thread is not explicitly controlled by the application, OSEK/VDX applications are dispatched by a deterministic scheduler. OSEK/VDX OS also provides standardized application interfaces (APIs), such as `ActivateTask`, `TerminateTask`, `SetEvent`, and `WaitEvent`, through which tasks can dynamically change task states, events, and the task execution order.

To ensure the reliability of OSEK/VDX applications, the behaviours of the OSEK/VDX scheduler should be taken into account in the verification process. Testing is an important checking technique used in industry, but it is difficult to exhaustively check OSEK/VDX applications using testing technique, especially because of incomplete test-cases. In addition, it is hard to reproduce an error trace when an error is detected by testing, due to the concurrency of OSEK/VDX applications.

The main difficulty is accuracy. Existing model checking methods for concurrent software usually check all of the interleavings of threads. If such methods are applied directly to OSEK/VDX applications, the verification process becomes too imprecise because the behaviours

of OSEK/VDX scheduler are not taken into account. As a result, a large number of unnecessary interleavings of tasks may be checked in the verification stage, which can lead to spurious bugs and increases the checking costs in terms of time and memory. Therefore, a more accurate bounded model checking approach for OSEK/VDX applications should take into account the deterministic scheduler, the ready queue, and the standardized application interfaces (APIs).

This report describes a CBMC-based checking approach that addresses this problem in two parts. First, the APIs invoked from tasks are taken into account during the verification process, so that the states of tasks, synchronization events, shared resources, and task execution order can be handled according to the behaviours of the OSEK/VDX scheduler. Second, the checking model is constructed to avoid exploring unnecessary behaviours in the verification stage.

The report makes three contributions:

- It describes how bounded model checking can be specialized for OSEK/VDX applications whose tasks are dispatched by a deterministic scheduler.
- It introduces a manifest for building a new checking model with interrupt-service-routine calls only at manifest-listed program locations.
- It presents benchmark results showing that the proposed approach reduces unnecessary task interleavings and removes spurious bugs reported by the baseline.

The rest of this report is structured as follows. Section 2 presents the OSEK/VDX execution model and the role of bounded model checking. Section 3 describes the proposed checking approach. Section 4 reports the benchmark results and discusses the accuracy and checking costs of the approach. Section 5 summarizes the main limitations of the current implementation.

2 Background

2.1 OSEK/VDX Task, Scheduler, and Event

An application developed on OSEK/VDX OS consists of multiple tasks [1]. OSEK/VDX OS can process two types of tasks: basic tasks and extended tasks. The states of a basic task are running, ready, and suspended. Compared with a basic task, an extended task can hold synchronization events and has an additional waiting state. OSEK/VDX OS also provides standardized application interfaces (APIs), and tasks within the application can invoke these APIs to dynamically change the states of tasks, synchronization events, and shared resources.

OSEK/VDX executes multiple tasks on one CPU using a deterministic scheduler based on static priorities and a ready queue.

Synchronization events are used to implement synchronous behaviours between tasks. Only extended tasks can hold events, and these events are used as criteria for switching task states from running state to waiting state or from waiting state to ready state. The event process module responds to three APIs: `SetEvent`, `WaitEvent`, and `ClearEvent`. For example, when a running task invokes `WaitEvent` to wait for an event, it cannot continue until that event is set by another basic or extended task through `SetEvent`.

2.2 Model Checking and Bounded Model Checking

Model checking is an efficient and exhaustive verification technique for checking whether a target system satisfies a given property. The key idea is to construct a model that describes the executions of the target system and then search the constructed model with the given property. If the property is violated, model checking reports a trace that shows how the error can occur. This trace is usually named a counterexample, and it helps developers reproduce the faulty behaviour.

Bounded model checking follows the same verification goal, but it restricts the checking process with a specified bound. Instead of generating the full state space at once, bounded model checking constructs a transition system within the bounded verification space and uses a solver to check whether the property is violated. If a violation is found, the solver can report a counterexample. This makes bounded model checking useful for systems with large state spaces while still preserving the practical value of an error trace.

2.3 Checking Approaches for OSEK/VDX Applications

The execution characteristics of OSEK/VDX applications make their verification different from general concurrent software verification. In OSEK/VDX applications, the running task is explicitly determined by the OSEK/VDX scheduler according to the ready queue and configuration data. At the same time, APIs invoked from tasks may change the scheduling of tasks, for example by causing a context switch or by leading to different task execution orders at different branches. Therefore, the key problem is how to construct an accurate checking model that represents the executions of the application and handles the behaviours of OSEK/VDX OS.

Existing checking approaches for OSEK/VDX applications can be summarized as follows:

- **Spin-based checking approach.** This approach constructs a combination model of the application model and the OSEK/VDX OS model. In this model, tasks within the application are represented as processes, while the OSEK/VDX OS model dispatches tasks and responds to the APIs invoked from tasks. This approach can accurately simulate the executions of OSEK/VDX applications in the real OSEK/VDX OS. However, its scalability is limited because the behaviours of the OSEK/VDX OS model are also explored in the verification stage, which may lead to state space explosion when the application invokes many APIs.
- **SMT-based bounded model checking approach.** This approach uses an execution path generator (EPG) to construct a transition system for the OSEK/VDX application. The OSEK/VDX OS model is embedded in the generation process to dispatch tasks and respond to the APIs invoked from tasks. Therefore, the OS model does not need to be explored directly in the verification stage. This approach is more efficient and scalable than the Spin-based checking approach for applications with many tasks and APIs. However, it may become inefficient when the application contains many branches, because many execution paths have to be explored to construct the transition system.
- **Sequentialization-based checking approach.** This approach translates an OSEK/VDX application into a sequential model and then applies existing model checkers such as Spin or CBMC to verify the generated model. During the sequentialization process, the scheduling behaviours of OSEK/VDX OS are explicitly performed, and the APIs invoked from tasks are handled during translation. This approach enables existing model checkers to verify OSEK/VDX applications by checking a sequential model instead of a concurrent model.

Among the checking approaches summarized above, this report follows the SMT-based bounded model checking direction. In this direction, the key work is to construct a transition system that correctly reflects the executions of the target application. For OSEK/VDX applications, this transition system must take into account the deterministic scheduler, the ready queue, and the APIs invoked from tasks. This report uses CBMC as the bounded model checker and refines the checking process so that OSEK/VDX-specific scheduling behaviours are represented before verification.

2.4 Baseline CBMC Architecture

CBMC is a bounded model checker for C programs [2]. Its checking process can be viewed in three main stages. First, the front-end and `goto-cc` translate C source files into a GOTO

program. Second, `goto-symex` symbolically executes the GOTO program and constructs an SSA-style equation. Third, the solver backend checks the generated equation and, when a property is violated, CBMC can reconstruct a counterexample trace.

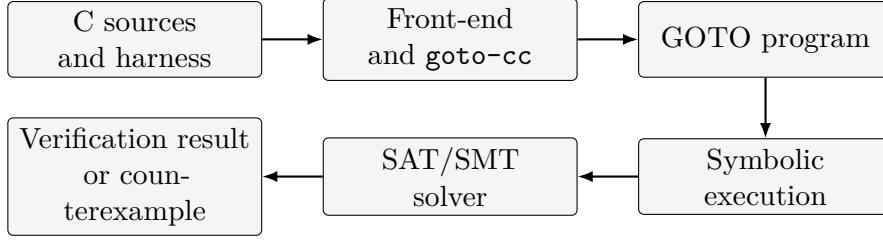


Figure 1: Baseline CBMC flow before the OSEK-aware and manifest-driven extensions described in this report.

This architecture is a strong baseline for C verification because bounded model checking can construct a transition system under a given bound and use a solver to check safety properties. However, this baseline is not sufficient by itself for OSEK/VDX applications. The limitation is not that CBMC lacks symbolic execution. The limitation is that the baseline CBMC flow does not automatically model the behaviours of the OSEK/VDX scheduler. In particular, APIs invoked from tasks can change task states, synchronization events, shared resources, and the ready queue. If these behaviours are not represented in the checking model, the verification process may not reflect the task execution order determined by the OSEK/VDX scheduler.

3 Methodology

3.1 Overview of the Proposed Approach

The method builds directly on the baseline CBMC architecture described in Section 2 and summarized in Figure 1. The goal is not to replace bounded model checking, symbolic execution, or the solver stage. The goal is to construct a more accurate checking model for OSEK/VDX applications before final verification. For ordinary C programs, the baseline CBMC flow is often sufficient. For OSEK/VDX applications, however, the checking model must also reflect how the deterministic scheduler, the ready queue, the task states, and the standardized application interfaces (APIs) determine task execution order.

This section explains the proposed approach in four parts. First, it explains how the checking model takes the OSEK/VDX scheduler into account. Second, it explains how the method reduces unnecessary interleavings. Third, it explains how those two ideas are combined to construct the final checking model. Fourth, it gives a short implementation note that maps the methodology to the current realization in this branch.

Table 1 provides the conceptual bridge from the baseline CBMC flow in Section 2 to the methodology developed in the rest of this section. The comparison focuses on the specific aspects that matter for OSEK/VDX verification: task execution order, standardized APIs, interruption-related behaviour, unnecessary interleavings, and the expected effect on verification accuracy.

The key point in Table 1 is that the proposed approach does not replace CBMC with a different verifier. It changes how the checking model is constructed before final verification so that the solver is applied to a model that better reflects the executions of OSEK/VDX applications.

To keep the discussion concrete, the remainder of this section uses one synthetic running example. *TaskControl* is a higher-priority task that waits for new sensor data. *TaskLogger* is a lower-priority task that can run while *TaskControl* is waiting. *ISR_Sensor* writes the shared

Aspect	Baseline CBMC	Proposed Approach
Task execution order	Follows ordinary program flow unless scheduler behaviour is encoded elsewhere.	Follows task states, the ready queue, and the deterministic scheduler inside the checking model.
Standardized APIs	Can be treated as ordinary function effects if no OSEK/VDX-specific model is added.	Are interpreted as operations that may change task states, events, the ready queue, and therefore the next running task.
Interruption-related behaviour	Broad asynchronous modelling can expose many possible interference points in the checking model.	Interruption-related behaviour is introduced only at program points where it can influence the checked execution.
Unnecessary interleavings	Irrelevant interleavings may remain in the checking model and enlarge the verification problem.	Unnecessary interleavings are reduced before final bounded model checking.
Expected accuracy outcome	May report a spurious bug or miss scheduler-dependent task execution order.	Aims to represent OSEK/VDX executions more faithfully while keeping CBMC's bounded model checking structure.

Table 1: Conceptual comparison between baseline CBMC and the proposed approach.

sensor state and signals the event awaited by *TaskControl*. The example is intentionally small, but it is large enough to show the three central decisions in the methodology: how task execution order is determined, how unnecessary interleavings are reduced, and how the final checking model is built.

3.2 Taking the OSEK/VDX Scheduler into Account

The first methodological step is to represent the behaviour of the OSEK/VDX scheduler inside the checking model. In an OSEK/VDX application, the next running task is not chosen by an unstructured interleaving of threads. It is chosen by a deterministic scheduler that depends on task states, priorities, events, and the ready queue. If the checking model ignores those elements, the resulting task execution order no longer matches the execution order of the application.

Listing 1 shows the running example used throughout this section. The example deliberately combines waiting, event signaling, shared-state update, and an unrelated local computation so that each later methodological step can be tied back to the same program.

Listing 1: Synthetic running example used throughout Section 3.

```

volatile int sensor_ready = 0;
volatile int sensor_value = 0;

TASK(TaskControl)
{
    EventMaskType observed = 0;
    WaitEvent(EVENT_SENSOR_READY);
    GetEvent(TASK_ID_Control, &observed);
    if ((observed & EVENT_SENSOR_READY) != 0u && sensor_ready) {
        int sample = sensor_value;
        assert(sample >= 0);
    }
    ClearEvent(EVENT_SENSOR_READY);
    TerminateTask();
}

```

Entity	Priority / type	Initial status and role
<i>TaskControl</i>	Higher-priority extended task	Running at the beginning of the path; waits for <code>EVENT_SENSOR_READY</code> ; later reads the event state and the shared sensor value.
<i>TaskLogger</i>	Lower-priority task	Ready while <i>TaskControl</i> is running; may execute while <i>TaskControl</i> is waiting; performs only local computation in this example.
<i>ISR_Sensor</i>	Interruption	Writes <code>sensor_ready</code> and <code>sensor_value</code> , then signals the event awaited by <i>TaskControl</i> .

Table 2: Configuration summary for the running example.

```

TASK(TaskLogger)
{
    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

ISR(ISR_Sensor)
{
    sensor_value = 7;
    sensor_ready = 1;
    SetEvent(TASK_ID_Control, EVENT_SENSOR_READY);
}

```

Table 2 summarizes the intended execution context for this example. The details are simple on purpose: *TaskControl* begins as the running task, *TaskLogger* is already ready at a lower priority, and the interruption can make the higher-priority task ready again by signaling the awaited event.

The central point is that the standardized APIs in Listing 1 cannot be treated as ordinary function effects. When *TaskControl* executes `WaitEvent`, the important effect is not simply that a call occurs. The important effect is that the task may move from the running state to the waiting state. Once that happens, *TaskLogger* may become the next runnable task. Later, when *ISR_Sensor* executes `SetEvent`, the event state changes and *TaskControl* may become ready again. Because *TaskControl* has higher priority, the deterministic scheduler may then dispatch it ahead of *TaskLogger*. The checking model must therefore represent state changes in the scheduler domain, not merely ordinary control-flow changes in the current C function.

Table 3 is the output of this first methodological step. It shows how the same source program leads to different verification states once the scheduler, the ready queue, and the task states are represented explicitly in the checking model.

Figure 2 summarizes the same causal chain at a higher level. The figure is not a control-flow graph. It is a compact view of how a standardized API changes task states and events, how those changes reach the ready queue, and how the deterministic scheduler turns that information into the next task execution step in the checking model.

Read clockwise, the cycle says the following. A standardized API such as `WaitEvent`, `GetEvent`, or `SetEvent` changes the scheduler-visible state of the application. Those changes update the task states and the event information associated with the affected tasks. The resulting state determines the contents of the ready queue. The ready queue is then interpreted by the deterministic scheduler to determine task execution order. That task execution order determines the next checking step, which may in turn lead to another standardized API or another scheduler-relevant state update. Baseline CBMC follows ordinary control-flow effects,

Step	Current context	Task states	Ready queue	Event state	Next dispatch decision
0	Start of path	<i>TaskControl</i> : running; <i>TaskLogger</i> : ready	<i>TaskLogger</i>	Clear	Keep <i>TaskControl</i> running
1	<i>TaskControl</i> executes WaitEvent while the event is clear	<i>TaskControl</i> : waiting; <i>TaskLogger</i> : ready	<i>TaskLogger</i>	Clear	Switch to <i>TaskLogger</i>
2	<i>TaskLogger</i> executes local computation	<i>TaskControl</i> : waiting; <i>TaskLogger</i> : running	Empty	Clear	Keep <i>TaskLogger</i> running unless interrupted
3	<i>ISR_Sensor</i> writes shared state and signals the event	<i>TaskControl</i> : ready; <i>TaskLogger</i> : ready or preemptable	<i>TaskControl</i> , <i>TaskLogger</i>	Set	Preempt with <i>TaskControl</i>
4	<i>TaskControl</i> resumes, observes the event, and reads shared data	<i>TaskControl</i> : running; <i>TaskLogger</i> : ready	<i>TaskLogger</i>	Set, then cleared	Keep <i>TaskControl</i> running

Table 3: State evolution for the running example when the checking model tracks task states, the ready queue, and the event state explicitly.

whereas the proposed approach must also update task state and scheduler state in the checking model.

In the running example, this means that the verification path should not simply continue with the syntactically next statement after **WaitEvent**. Instead, the checking model should represent that *TaskControl* is blocked, that *TaskLogger* may run while the higher-priority task is waiting, and that *ISR_Sensor* can make *TaskControl* ready again. Only then does the task execution order in the verification result reflect the execution order defined by the OSEK/VDX scheduler.

3.3 Reducing Unnecessary Interleavings

Taking the scheduler into account is not sufficient by itself. The second methodological step is to reduce unnecessary interleavings in the checking model. Existing checking approaches for concurrent software often consider a very broad set of asynchronous behaviours. In the present setting, that can enlarge the verification problem and can also produce a spurious bug when interruption-related behaviour is introduced at program points that are irrelevant to the property being checked.

The key question is therefore not “where could an interruption happen in general?” The key question is “where can interruption-written state affect the checked execution?” This is a narrower and more useful question because it focuses the checking model on observable effects. If interruption-written state cannot influence a later branch, a task-state transition, or a property-relevant shared-data read, then representing interruption-related behaviour at that point adds interleavings without improving accuracy.

The running example makes this distinction concrete. *ISR_Sensor* writes the event state observed by *TaskControl* and the shared variables **sensor_ready** and **sensor_value**. Those writes matter near the points where *TaskControl* decides whether it should block, where it observes the signaled event, where it branches on that information, and where it reads the shared sensor value. By contrast, the two checksum updates inside *TaskLogger* are purely local in this

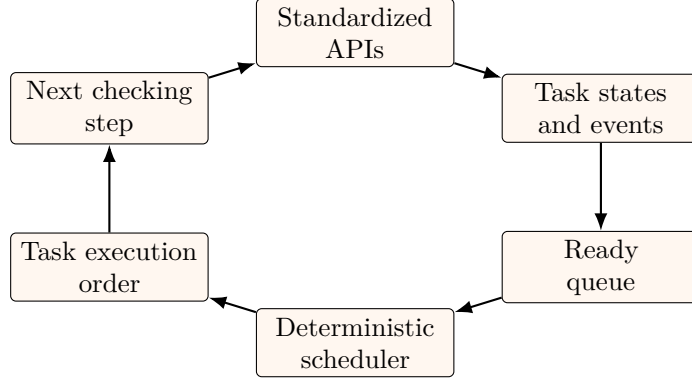


Figure 2: Conceptual cycle from standardized application interfaces (APIs) to task states, the ready queue, the deterministic scheduler, and the next checking step.

example. They do not observe the event state, they do not change the ready queue, and they do not influence the later branch or shared-data read that matter for the checked path.

Table 4 shows the output of this second methodological step. The table is intentionally human-readable. It does not list raw artifact fields. It records only the conceptual question answered by the analysis: which program points must retain interruption-related behaviour because they can affect the checked execution?

The point of Table 4 is not to predict the exact moment of every interruption. The point is to keep only those program points where interruption-written state can change the path explored by bounded model checking. Baseline CBMC with broad asynchronous modelling may still explore both the kept and the discarded points. The proposed approach keeps the first group and removes the second group from the final checking model, which reduces unnecessary interleavings without discarding the scheduler-relevant behaviour.

3.4 Building the Final Checking Model

The proposed approach combines the previous two ideas by separating two decisions that are often mixed together. The first decision is analytical: identify the program points at which interruption-related behaviour can influence the checked execution. The second decision is constructive: build a new checking model that introduces interruption-related behaviour only at those selected points.

This separation matters because the two decisions answer different questions. The analytical step asks where interruption-written state can affect task execution order or a property-relevant computation. The constructive step asks how to represent those effects in the final checking model. Keeping those steps separate avoids a monolithic transformation in which relevance and model rebuilding are entangled. As a result, the final checking model can remain narrower than a blanket insertion of interruption-related behaviour at many unrelated points.

For the running example, the analytical result can be summarized in the compact form shown in Listing 2. This summary is the bridge between the two stages: it is already specific enough to guide reconstruction, but it is still expressed in terms of the checked path rather than in low-level artifact details.

Listing 2: Human-readable summary of the selected program points for the running example.

Selected points 1. Before <code>WaitEvent(EVENT_SENSOR_READY)</code> 2. Before <code>GetEvent(TASK_ID_Control, &observed)</code> 3. Before the branch that checks <code>observed</code> and <code>sensor_ready</code> 4. Before the read of <code>sensor_value</code>

Program point in the running example	Depends on interruption-written state?	Keep / discard	Reason
Immediately before <code>WaitEvent(EVENT_SENSOR_READY)</code>	Yes	Keep	The interruption can determine whether the task remains running or moves to the waiting state.
Immediately before <code>GetEvent(TASK_ID_Control, &observed)</code>	Yes	Keep	The returned event mask depends on whether the interruption has already signaled the awaited event.
Immediately before the branch that checks <code>observed</code> and <code>sensor_ready</code>	Yes	Keep	The branch condition depends directly on state written by the interruption.
Immediately before the read of <code>sensor_value</code>	Yes	Keep	The shared value read by <i>TaskControl</i> is produced by the interruption.
Immediately before <code>checksum = checksum + 1</code> in <i>TaskLogger</i>	No	Discard	The computation is local to <i>TaskLogger</i> and cannot observe interruption-written state in this example.
Immediately before <code>checksum = checksum * 2</code> in <i>TaskLogger</i>	No	Discard	This step is also local and does not affect task release, event state, or the later shared-data read.

Table 4: Conceptual selection result for interruption-relevant program points in the running example.

Excluded points - The local checksum updates inside <i>TaskLogger</i>
--

Figure 3 shows how the final checking model is then built. The upper lane shows the baseline CBMC path. The lower lane shows the additional construction stages used by the proposed approach before final bounded model checking.

Conceptually, the upper lane starts from the original program and proceeds directly to model construction, symbolic execution, and bounded model checking. The lower lane inserts four additional responsibilities. The first added block builds a project-scoped view of the program rather than a single straight-line execution view. The second added block identifies where interruption-written state can affect the checked path. The third added block rebuilds a restricted source-level checking model that retains interruption-related behaviour only at those selected points. The fourth added block applies scheduler-consistent symbolic execution so that standardized APIs change task states, the ready queue, and task execution order in a way that matches OSEK/VDX semantics. This is the second major difference from baseline CBMC: the proposed approach changes both where interruption-related behaviour appears and how the resulting task execution order is interpreted during verification.

Listing 3 shows the output of the constructive step for the running example. The guarded calls are part of the checking model, not part of the original application program. They are placed only at the selected points. No such call is inserted inside the local checksum updates of *TaskLogger*, because those updates were excluded as unnecessary interleavings in Table 4.

Listing 3: Illustrative restricted checking model for the running example.

<pre> TASK(TaskControl) { EventMaskType observed = 0; if (nondet_bool()) ISR_Sensor(0); WaitEvent(EVENT_SENSOR_READY); </pre>

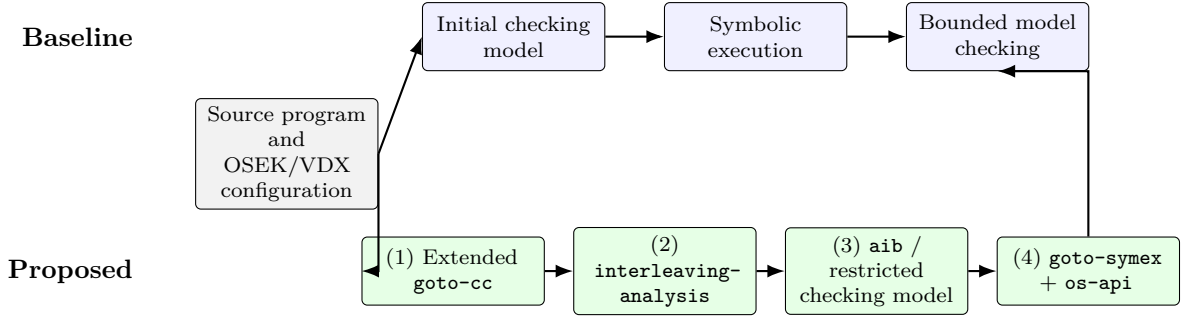


Figure 3: Baseline and improved checking-model construction. Numbered blocks indicate stages added by the proposed approach.

```

if (nondet_bool()) ISR_Sensor(0);
GetEvent(TASK_ID_Control, &observed);
if (nondet_bool()) ISR_Sensor(0);
if ((observed & EVENT_SENSOR_READY) != 0u && sensor_ready) {
    if (nondet_bool()) ISR_Sensor(0);
    int sample = sensor_value;
    assert(sample >= 0);
}
ClearEvent(EVENT_SENSOR_READY);
TerminateTask();
}

TASK(TaskLogger)
{
    int checksum = 0;
    checksum = checksum + 1;
    checksum = checksum * 2;
    TerminateTask();
}

```

The methodological role of Listing 3 is straightforward. The checking model still admits interruption-related behaviour, but only at points where that behaviour can change the execution being checked. This preserves the main advantage claimed in Sections 1 and 2: fewer unnecessary interleavings without ignoring the OSEK/VDX scheduler or the shared state written by interruptions.

3.5 Implementation Note

The current branch realizes this methodology with a small set of focused extensions rather than a monolithic verifier rewrite. Table 5 summarizes the role of each concrete component and the artifact or checking-model effect it produces.

Figure 3 uses these component boundaries implicitly. The added blocks in the lower lane correspond, in order, to the project-scoped model-construction extension around `goto-cc`, the separate `interleaving-analysis` stage, the `aib` rewriting step built on `lib-interleaving-block`, and the `goto-symex/os-api` realization of OSEK/VDX API effects during symbolic execution.

Exact command-line spellings, manifest-field names, and concrete artifact formats are kept out of the methodology body and collected in Section A. That appendix is the right place for interface-level details once the theoretical flow in this section is understood.

Component	What it consumes and does	What it produces
<code>goto-cc</code> extension	Consumes the source program and configuration data, then builds the initial checking model while preserving the project-level view needed by the later analysis stage.	An initial checking model together with the project-scoped information needed to analyze interruption relevance.
<code>interleaving-analysis</code>	Consumes the initial checking model and the designated interruption sources, then identifies program points where interruption-written state can affect the checked path.	A selection artifact describing the retained program points for restricted reconstruction.
<code>aib</code> with <code>lib-interleaving-block</code>	Consumes the original project tree and the selection artifact, then rebuilds a restricted source-level checking model with guarded interruption calls only at the retained points.	A rewritten project view that is narrower than blanket interruption insertion.
<code>goto-symex</code> with <code>os-api</code>	Consumes the restricted checking model and the OSEK/VDX configuration, then realizes the effects of standardized APIs on task states, events, the ready queue, and dispatch decisions during symbolic execution.	Scheduler- consistent symbolic states for final bounded model checking.

Table 5: Concrete realization of the methodology in the current branch.

4 Evaluation

This section uses only the public benchmark families in the repository. Internal benchmark families are intentionally excluded. The goal is not to claim that the revised flow is always faster than the stock baseline. The checked-in evidence shows the opposite in many cases because the manifest, injection, and rebuild stages add cost. The evaluation instead answers two narrower questions:

1. Do the public benchmark artifacts contain cases where the revised flow is judged more faithful than the stock asynchronous baseline?
2. What cost is paid for that increase in accuracy?

4.1 Public Benchmark Scope

The checked-in public report `check-src/benchmark.md` covers the five suites summarized in Table 6. Together they contribute 43 measured cases.

4.2 Public Correctness Summary

The public subset has 43 measured cases. The correctness counts in Table 7 are recomputed for that public subset only.

Two points stand out. First, the improved pipeline is not merely different from the stock baseline; it is often audited as more faithful. Second, the benchmark report explicitly keeps manual-review categories instead of silently forcing every mismatch into a win/loss label. That is important engineering hygiene for an evolving verification branch.

Suite	Cases	Headline-ready	Notes
icbmc-large	12	yes	Larger i-CBMC-derived public cases.
icbmc-timeout-30m	4	no	Public diagnostic cases kept outside the automatic suite.
icbmc-timeout-30m-blink-highmem	2	no	Public high-memory blink cases.
icbmc-timeout-30m-logger-stable	4	no	Public logger-focused diagnostic cases.
intabs-large	21	yes	Larger public IntAbs-derived cases.

Table 6: Public benchmark suites used in this section.

Benchmark verdict	Cases	Interpretation
same_outcome_comparable	17	Stock and improved reached the same verification class.
improved_correct	10	The improved pipeline is audited as the more faithful model for the case.
no_injection_false	6	The improved path reported no injection candidates even though state written by an interleaving function can still affect the main-reachable state.
needs_manual_review	3	Output mismatch exists, but the current audit is not strong enough to choose a correct variant.
pending_manual_review	7	Mismatch exists and no verdict rule has yet classified the case.

Table 7: Correctness summary for the public benchmark subset only.

4.3 Representative Public Cases

Three public cases illustrate the different ways in which the revised flow changes the verification result.

icbmc-timeout-30m-logger-stable / logger2-bug-conc-cprover. The stock baseline reports *VERIFICATION FAILED* in 56 233 ms with 298.2 MB peak RSS, while the improved pipeline reports *VERIFICATION SUCCESSFUL* in 2017 ms with 16.6 MB peak RSS. The checked-in audit explains the difference directly: the stock async baseline exposes a non-atomic asynchronous interleaving in the logger update, whereas the improved pipeline checks the extracted interruption entry as one atomic region. This is a clear public example of unnecessary interleavings producing a spurious bug.

intabs-large / i8xx-tco-1. The stock baseline reports *VERIFICATION FAILED* in 1202 ms with 13.1 MB peak RSS, while the improved pipeline reports *VERIFICATION SUCCESSFUL* in 3395 ms with 15.3 MB peak RSS. The benchmark audit again points to unnecessary interleavings in the stock model. The stock run allows a non-atomic asynchronous interleaving, whereas the improved pipeline keeps the interruption entry atomic and narrows the candidate set to three guarded interleavings. This case shows the main accuracy gain of the revised flow even though the full measured time grows from 1202 ms to 3395 ms.

intabs-large / sc520wdt-2. The stock baseline reports *VERIFICATION SUCCESSFUL* in 1172 ms with 41.0 MB peak RSS. The improved pipeline reports *VERIFICATION FAILED* in 3559 ms with 16.7 MB peak RSS. Here the improved pipeline injects closer/writer interleavings and reaches a violated property that the stock baseline proves away. This is the opposite public pattern from the previous two cases: the revised flow is slower, but it exposes an error trace that is not reached by the stock model. The full measured time rises from 1172 ms to 3559 ms, but the peak RSS drops from 41.0 MB to 16.7 MB.

Taken together, these public cases show the main cost of the revised flow. Even when the

verification phase itself is not dramatically slower, the extra manifest, injection, and rebuild phases raise the full measured time. The public evidence therefore supports a clear trade-off: the revised flow aims at a more accurate verification result, not at a faster one.

For the use case of this report, that trade is acceptable. The objective is not to outperform the stock baseline on total wall-clock time, but to reduce unnecessary interleavings and make bounded model checking more accurate for OSEK/VDX applications.

5 Discussion and Limitations

The revised flow is deliberately conservative in some places and intentionally incomplete in others. Those limitations are not incidental; they define where future work should focus.

5.1 What the Current Design Gets Right

The strongest engineering choice in the branch is the separation between the behaviours of OSEK/VDX scheduler and the placement of interruption-related instrumentation. The task execution order is determined during verification, while the candidate program points are determined by an explicit analysis step and recorded in a manifest. This keeps the method maintainable. A change to OSEK/VDX scheduling policy does not automatically require a change to the instrumentation step, and a change to the candidate-line heuristics does not automatically require a change to the verification core.

The evidence in Section 4 also suggests that the extra structure is not just architecturally neat; it matters to outcomes. The benchmark report contains multiple cases in which the improved pipeline is audited as the more faithful model even though it is slower.

5.2 Current Analysis Limits

- **Source-file-driven target discovery.** Interleaving functions are selected from an explicit source-file list. If the relevant interruption file is not part of the build, the analysis cannot discover it.
- **Global-write-centric conflict rule.** Candidate lines are derived from globals written by the interleaving function and read or written by the main verification path. This keeps the rule simple, but it ignores purely read-side interactions by the interleaving function.
- **.c-only candidate filtering under project scope.** When project-root filtering is enabled, candidate recording is restricted to .c files under the root. Generated or indirect artifacts outside that slice are ignored.
- **Hand-written manifest IO.** Both the manifest writer and the `aib` reader are hand-coded. There is no schema version field, which makes coordinated changes brittle.

5.3 Current OSEK Model Limits

- **Partial OIL subset.** The parser currently handles the fields needed by the implemented scheduling rules: `TASK`, `PRIORITY`, `SCHEDULE`, `AUTOSTART`, `TYPE`, and `EVENT_MASK`.
- **AUTOSTART is metadata, not boot behavior.** The configuration records `AUTOSTART`, but the current flow still expects explicit task activation from the harness or runtime stubs.
- **WaitEvent is an approximation.** The implementation models the block / no-block decision and wake-up dispatch, but not every possible continuation detail of the full operating-system behaviour.

- **API coverage is intentionally narrow.** The current branch focuses on task and event services relevant to the public benchmarks. Resource, alarm, and interruption-related APIs beyond this subset are not yet modeled through the same boundary.

5.4 Performance Trade-offs

The benchmark report consistently shows that the revised flow pays an overhead tax. Even public cases add a manifest phase, an injection phase, and a rebuild. Larger cases can still be worth that cost when the stock baseline either misses a real bug or invents one through unnecessary interleavings, but the branch is not yet an optimization pass.

This suggests a practical reading of the current status: the branch is already useful as a more accurate bounded model checking workflow, but not yet as a drop-in faster replacement for stock CBMC. Improving analysis precision without paying so much orchestration cost is a natural next step.

6 Conclusion

This report documented a CBMC-based bounded model checking flow for OSEK/VDX applications. The central problem is accuracy: if the deterministic scheduler is ignored, many unnecessary interleavings of tasks may be superfluously checked and may result in a spurious bug. The revised flow addresses that problem by taking the behaviours of OSEK/VDX scheduler into account and by restricting interruption-related instrumentation through an explicit manifest.

The checked-in public benchmark evidence supports that choice. The revised flow is often slower because it adds analysis, instrumentation, and rebuild phases, but it is audited as the more faithful model in meaningful public cases. In particular, it removes unnecessary interleavings that produce spurious bugs and reaches some error traces that are not reached by the stock baseline.

The remaining work is also clear. The supported OSEK/VDX subset can be widened, the manifest contract can be made more robust, and the candidate-line rule can be made less dependent on a simple global-write heuristic. Even in its current form, however, the branch demonstrates a practical path toward more accurate bounded model checking for OSEK/VDX applications.

A Appendix: Concrete Interfaces

A.1 OSEK/VDX Checking Options

The methodology in Section 3 refers to OSEK/VDX-specific model construction without listing its command-line spellings. The following schematic command shows the documented options for selecting the OSEK/VDX checking model and the corresponding OIL configuration file.

Listing 4: Schematic OSEK/VDX option usage.

```
cbmc main.c --os-api osek --osek-oil app.oil
```

A.2 Example goto-cc Invocation

The following command illustrates the public project-mode flow on the `i8xx-tco-1` benchmark tree. It records the project root, names the interruption source file explicitly, and emits an interleaving manifest.

Listing 5: Example project-mode interleaving analysis command.

```
goto-cc \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/main.c \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/isr
.c \
--interleaving-project-root \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \
--interleaving-source-files \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline/isr_define/
isr.c \
--interleaving-output \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \
-o check-src/benchmark-sources/intabs/cases/i8xx-tco-1/i8xx-tco-1.out
```

A.3 Example aib Invocation

Listing 6: Example AIB rewrite command.

```
aib \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/improved-pipeline \
check-src/benchmark-sources/intabs/cases/i8xx-tco-1/interleaving_pipeline.json \
out/i8xx-tco-1_injected \
out/i8xx-tco-1_injected/interleaving_pipeline_effective.json
```

A.4 Illustrative Guarded Call

The restricted checking model is created by inserting guarded interruption calls only at the selected program points. The example below is illustrative rather than benchmark-specific.

Listing 7: Illustrative guarded interruption call.

```
if (nondet_bool()) isr1(0);
```

A.5 Documented Selection-Artifact Fields

Field	Meaning
project.project_root	Root directory used for path scoping and report-path relativization.
project.interleaving_source_files	Source files whose functions are treated as interleaving functions.
project.translation_units	Effective build inputs associated with the initial checking model that produced the selection artifact.
interleaving[*].name	Interleaving function identifier recorded for rewriting.
interleaving[*].write_global_var	Global symbols that the interleaving function may write.
interleaving[*].line_added_block_with_file	Project-relative file and line pairs where guarded calls may be inserted.

Table 8: Documented fields of the selection artifact used between analysis and rewriting.

A.6 Manifest Excerpt

The manifest below is a schematic excerpt showing only the documented public fields. It illustrates how project metadata and interleaving candidates are separated.

Listing 8: Schematic manifest excerpt.

```
{
  "project": {
    "project_root": "/path/to/check-src/benchmark-sources/intabs/cases/i8xx-tco-1/
improved-pipeline",
    "interleaving_source_files": ["isr_define/isr.c"],
    "translation_units": [
      "main.c",
      "isr_define/isr.c"
    ]
  },
  "interleaving": [
    {
      "name": "isr1",
      "write_global_var": ["<global_1>", "<global_2>"],
      "line_added_block_with_file": [
        { "file": "main.c", "line": [<line_a>, <line_b>] }
      ]
    }
  ]
}
```

This schema is the compatibility boundary between `interleaving-analysis` and `aib`; any field change must be implemented on both sides.

References

- [1] *OSEK/VDX Operating System Specification 2.2.1*. Document OS221.doc, January 16, 2003. OSEK Group. 2003.
- [2] Diffblue Ltd. *CBMC Architecture*. 2026. URL: <https://diffblue.github.io/cbmc/cbmc-architecture.html> (visited on 06/19/2026).